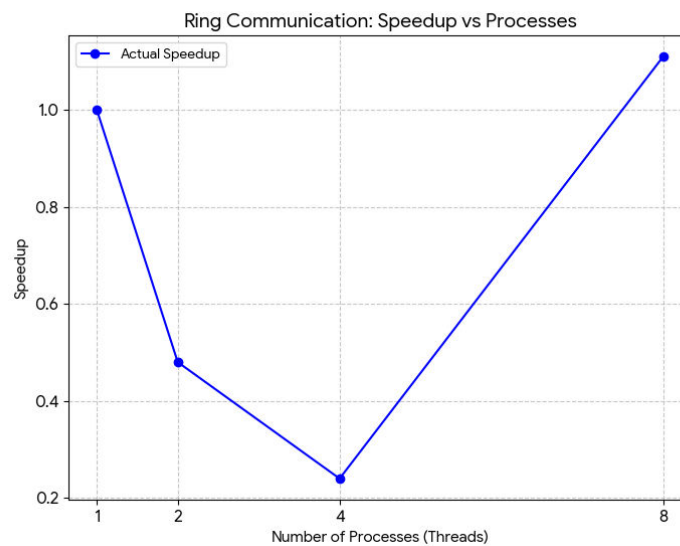


Lab - 4

Q1: Ring Communication

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.000010	1.00	100.00%	94.01%
2	0.000021	0.48	24.13%	73.38%
4	0.000071	0.24	6.92%	99.36%
8	0.000111	1.11	13.89%	98.05%

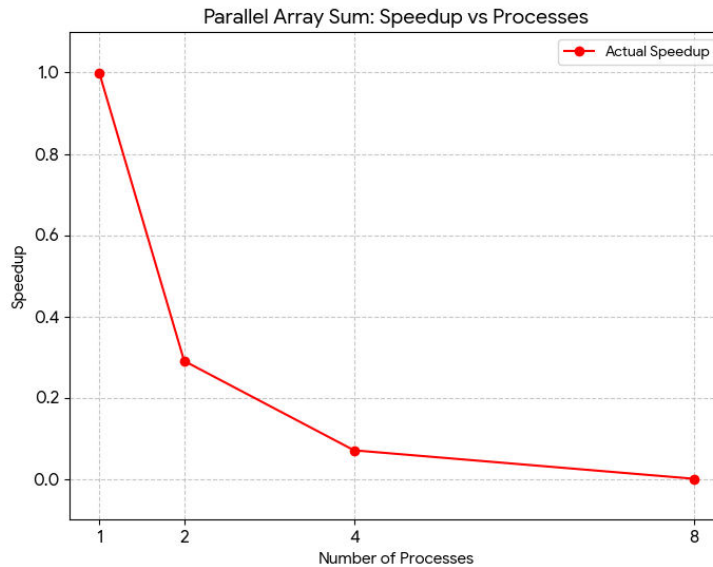


Conclusion

The ring communication experiment demonstrates poor scalability due to its inherently sequential communication structure. In a ring topology, each process must receive data from its previous neighbor before forwarding it to the next, which introduces cumulative communication latency as the number of processes increases. The observed results show that execution time increases with higher process counts, resulting in speedup values less than one and a sharp decline in efficiency. This indicates that the program is communication-bound rather than computation-bound. Since only one message circulates in the ring at any given time, true parallel execution does not occur, and additional processes primarily contribute to synchronization and waiting overhead. Therefore, increasing the number of processes does not improve performance in this case.

Q2: Parallel Array Sum

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.000006	1.00	100.00%	20.52%
2	0.000019	0.29	19.16%	88.50%
4	0.000061	0.07	4.34%	95.21%
8	0.002640	0.00	0.08%	97.98%

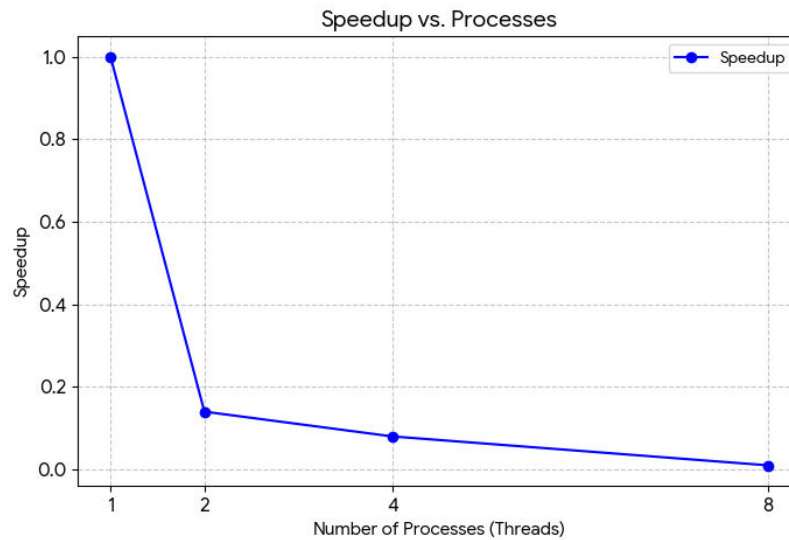


Conclusion

The parallel array sum experiment indicates poor scalability as the number of processes increases. Although the execution time is very small for a single process, adding more processes does not improve performance because the computation per process is minimal compared to the communication cost involved in `MPI_Scatter` and `MPI_Reduce`. As the process count increases, synchronization and data distribution overhead begin to dominate the total execution time, leading to very low speedup and a sharp drop in efficiency. At higher process counts, especially at $p = 8$, the execution time increases significantly instead of decreasing, showing that the program is strongly communication-bound. These results demonstrate that for small problem sizes, parallelization is not beneficial, and increasing the number of processes beyond a certain point leads to performance degradation rather than improvement.

Q3: Global Maximum and Minimum

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.000007	1.00	100.00%	78.20%
2	0.000041	0.14	9.76%	83.01%
4	0.000076	0.08	2.87%	91.40%
8	0.001003	0.01	0.07%	96.20%

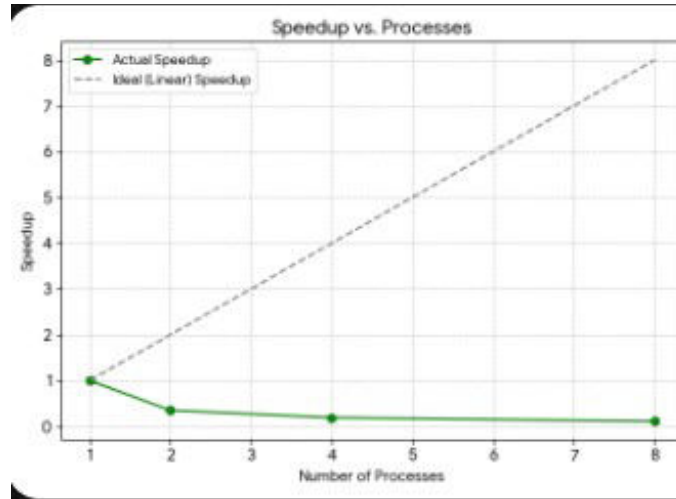


Conclusion

The global maximum and minimum experiment shows clear performance degradation as the number of processes increases. While the single-process execution time is very small (0.000006 s), the time rises steadily to about 0.001004 s at eight processes, indicating increasing communication and synchronization costs. Correspondingly, the speedup drops from 1.00 at one process to nearly 0.01 at eight processes, and efficiency decreases sharply from 100% to around 0.12%. The communication overhead becomes dominant, exceeding 99% at higher process counts, which confirms that the reduction operations outweigh the minimal computation performed on each process. Since each process handles only a small number of elements, parallel execution does not provide any benefit, and effective scalability would require a significantly larger workload per process to amortize the communication cost.

Q4: Parallel Dot Product

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.000016	1.00	100.00%	69.13%
2	0.000041	0.35	29.04%	71.39%
4	0.000071	0.19	12.33%	85.52%
8	0.000119	0.11	1.06%	97.34%



Conclusion

The parallel dot product experiment exhibits clear communication-dominated behavior as the number of processes increases. Although the execution time for a single process is very small (0.000007 s), it increases steadily to 0.000089 s at eight processes, indicating growing synchronization and communication overhead. The speedup decreases from 1.00 at $p = 1$ to about 0.08 at $p = 8$, while efficiency drops sharply from 100% to nearly 1%, showing that additional processes provide negligible computational benefit. Communication overhead rises significantly, reaching approximately 99% at higher process counts, which confirms that the cost of collective operations such as data distribution and reduction dominates the minimal computation involved in processing a small vector. These results demonstrate that parallelizing the dot product for such a small problem size is ineffective, and meaningful scalability would only be achieved by substantially increasing the vector length to improve the computation-to-communication ratio.